

HE-IFD: Privacy-Preserving One-Shot Federated Distillation in a Shared Loss Basin under Homomorphic Encryption

Halil İbrahim Kanpak , Sinem Sav , Alptekin Küpçü 

Abstract—Federated learning lets parties train a shared model without pooling their data, but its one-shot variant, which communicates in a single round, faces an unresolved privacy dilemma: sending a client’s contribution in plaintext exposes it, while protecting it with differential privacy injects noise that degrades the released model and offers only a statistical, not a cryptographic, guarantee, and homomorphic encryption has so far been confined to multi-round encrypted training or to secure-aggregation primitives. We present HE-IFD, a one-shot federated distillation whose contributions are protected by homomorphic encryption. Each client distills its local teacher into a small shared classifier head along a bounded trajectory, and the server combines the resulting parameter displacements with a single sample-weighted linear operation under multiparty CKKS, at multiplicative depth one. The key to making a one-shot linear aggregation work under heterogeneous data is a shared loss basin: a common, possibly weak, initialization agreed from cheap per-class summaries, whose role is to align the clients’ updates rather than to classify well. Across vision and language tasks, and from small networks to frozen pretrained backbones, the method produces a common model stronger than any client’s own and close to a centralized one, while giving each client coverage of classes it never observed locally. A real multiparty CKKS implementation reproduces the plaintext aggregate within the approximation bounds of CKKS at a few megabytes per round, and differentially private basin summaries incur negligible accuracy cost across the practical range of privacy budgets.

Index Terms—Federated learning, one-shot federated learning, knowledge distillation, homomorphic encryption, multiparty computation, differential privacy.

I. INTRODUCTION

Many organizations would benefit from training a shared model on their combined data but cannot pool it, whether for legal, competitive, or privacy reasons. Federated learning addresses this by exchanging model updates instead of data, but in its usual form it does so over many rounds of communication, and each round discloses a fresh update computed on private data. One-shot federated learning removes the first cost by collapsing the entire interaction into a single upload and

download. It does not, by itself, remove the second. A client’s single contribution still encodes its private data, and the field’s answers to this have been unsatisfying at the extremes: sending the contribution in plaintext exposes it, while protecting it with differential privacy injects noise into the released model and provides only a statistical, not a cryptographic, guarantee. Homomorphic encryption promises a way out by operating on contributions while they stay encrypted, but it has been applied to federated learning either as multi-round encrypted training, whose cost is measured in hours and gigabytes, or as a secure-aggregation primitive embedded in an iterative protocol. No existing method delivers a one-shot federated distillation whose contributions are protected cryptographically rather than by accuracy-degrading noise.

We present **HE-IFD**. Each client distills its own locally trained teacher into a small shared classifier head, and the server combines the clients’ contributions with a single linear operation, $\theta^* = \theta_0 + \sum_j w_j \Delta_j$, evaluated under multiparty CKKS. The server holds the shared initialization θ_0 as a public constant and never modifies it during the protocol: it only adds the aggregated displacement to this preserved model, so no training takes place on the server side. Because the combination is linear, it maps onto the operations a homomorphic scheme evaluates at multiplicative depth one, without bootstrapping, and moves a few megabytes per round rather than hundreds of gigabytes. The privacy of the contributions is therefore cryptographic, and because the feature extractor is frozen and never enters the encrypted combination, the same protocol and the same cost carry from a small from-scratch network to a frozen pretrained backbone.

The difficulty a one-shot linear aggregation must overcome is that independently trained models do not average well under heterogeneous data: starting from different initializations and fitting different local distributions, their updates point in conflicting directions and cancel when summed. We resolve this with what we call a shared loss basin. Before any local training, the clients agree on a common initialization for the head, built cheaply from per-class feature summaries exchanged over peer-to-peer channels. This initialization need not be an accurate classifier; its role is to place every client in a common region of the loss landscape so that their subsequent bounded updates are expressed in one frame and reinforce rather than cancel. A weak but shared starting point is worth

Corresponding Author: Sinem Sav (sinem.sav@cs.bilkent.edu.tr)

Halil İbrahim Kanpak is with the Department of Computer Engineering, Koç University, İstanbul, Türkiye (e-mail: hkanpak21@ku.edu.tr).

Sinem Sav is with the Department of Computer Engineering, Bilkent University, Ankara, Türkiye (e-mail: sinem.sav@cs.bilkent.edu.tr).

Alptekin Küpçü is with the Department of Computer Engineering, Koç University, İstanbul, Türkiye (e-mail: akupcu@ku.edu.tr).

Manuscript received XX XX, XXXX; revised XX XX, XXXX.

more than a strong but private one; and the protocol still produces a usable model with no shared initialization at all, the basin serving to improve it further under heterogeneity.

We show across vision and language tasks, and from small networks to frozen vision transformers and sentence encoders, that the protocol produces a common model stronger than any client's own and close to a centralized model trained on the pooled data, while every client gains the ability to classify categories it never observed locally. With a frozen pretrained backbone the explicit agreement step is not even needed: the fixed features already place all clients in a common loss basin, so the shared initialization comes for free. We confirm in a real multiparty CKKS implementation that the encrypted aggregation reproduces the plaintext result within the approximation bounds of CKKS, at a cost of a few megabytes per round, and we trace the privacy–utility frontier of the basin summaries to show that their accuracy cost is negligible across the single-digit privacy budgets typical of differentially private learning.

Our contributions are as follows.

- We introduce, to our knowledge, the first one-shot federated distillation whose contributions are protected by homomorphic encryption, with a single server operation of multiplicative depth one under multiparty CKKS (section V).
- We identify the shared loss basin as the mechanism that makes a one-shot linear aggregation succeed under heterogeneity, and show that the basin's value lies in aligning client updates rather than in classifying well, so that even a deliberately weak or privacy-protected basin suffices, and a frozen pretrained backbone supplies the basin at no cost (sections VI-C and VI-D).
- We give a privacy design that protects the model updates with cryptography, adding no perturbation to them, and confines differential privacy to small per-class summaries, and show empirically that the resulting privacy–utility frontier is flat across practical privacy budgets (sections V-E and VI-G).
- We demonstrate that the method yields a common, superior model across from-scratch and frozen pretrained backbones in both modalities, and validate the encrypted aggregation in a real multiparty CKKS implementation at a cost of a few megabytes per round (sections VI-B and VII).

II. RELATED WORK

The setting we study sits at the meeting point of three lines of work that have largely developed in isolation: how federated learning communicates, how it is made private, and how it has been compressed into a single round. Tracing how each arrived at its present state makes clear both what is established and where the gap we address lies.

Federated learning began as an iterative procedure. In the original formulation a server repeatedly broadcasts a global model, clients improve it on their local data, and the server averages the returned updates [1]. This works well but inherits two persistent costs from its iterative nature. The first is

communication: reaching a good model can take tens or hundreds of rounds, each moving a full model in both directions. The second is exposure: every round reveals a fresh update computed on private data, and a long sequence of such updates is a rich target for inference. Much of the subsequent literature can be read as attempts to control one of these two costs without worsening the other.

The exposure problem has been met with two broad tools, distinguished by whether they alter the model that is ultimately released. Secure aggregation lets a server learn only the sum of the clients' updates and nothing about any individual one, using masking protocols that cancel in the aggregate [2]; it is lossless, in that the aggregate is exact, but it protects only the per-round sum and still operates within the iterative protocol. Differential privacy instead perturbs the updates themselves, most commonly through differentially private stochastic gradient descent [3]; the guarantee is strong and composable, but because the noise enters the released model it trades privacy directly against accuracy. This distinction, between privacy that costs accuracy and privacy that does not, is the axis along which our own privacy design is best understood, and it recurs throughout what follows.

Homomorphic encryption offers a third route, protecting the updates by computing on them while they remain encrypted. In its most ambitious form it has been used to train a neural network entirely under encryption across many parties. POSEIDON, for instance, runs encrypted stochastic gradient descent under multiparty CKKS, with the nonlinear activations replaced by polynomial approximations so that they can be evaluated homomorphically [4]. The privacy adds no accuracy cost and the result is strong, but the approach remains fundamentally iterative and pays for the encrypted nonlinearities with a deep multiplicative circuit, repeated bootstrapping, and a wall-clock measured in hours. A lighter line keeps the encryption but narrows its job to aggregation alone: schemes that combine multi-key CKKS with masking reduce a round of secure summation to a single non-interactive client-to-server transmission [5]. These are efficient and genuinely cryptographic, but they encrypt the sum of model updates within an iterative training protocol rather than a one-shot contribution, and they perform aggregation rather than knowledge transfer. On the encryption axis these works are the closest to ours, and the contrast is precise: they are either multi-round or are secure-aggregation primitives, whereas we encrypt a single distillation displacement.

Homomorphic encryption has also been applied to federated learning purely for aggregation, encrypting the per-round updates so the server can sum them without seeing any in the clear; BatchCrypt, FedML-HE, and FedSHE develop this for cross-silo training with various packing and quantization optimisations [6], [7], [8]. These remain multi-round and, like secure aggregation, protect the sum rather than transferring knowledge. A recurring obstacle across all encrypted-training work is the nonlinearity: activations must be replaced by polynomial approximations to be evaluated homomorphically, which deepens the multiplicative circuit and is delicate to train, as a line of work on polynomial activations and encryption-friendly networks documents [9], [10], [11], [12], [13]. Our

design sidesteps this entirely: the only encrypted operation is linear, so no activation is ever evaluated under encryption and the circuit stays at depth one.

The communication problem has driven the parallel development of one-shot federated learning, which collapses the entire exchange into a single upload and download [14], [15]. The earliest approaches transferred knowledge rather than weights, having clients exchange predictions on a shared public set so that heterogeneous models could teach one another [16], [17]; in practice these remained iterative, repeating the exchange to converge. Ensemble distillation made the distillation step explicit by training a server model against the clients' aggregated predictions, though in its standard configuration it too runs for many rounds [18]. The decisive step to a true single round came with data-free one-shot methods, which avoid any shared data by having the server distil the clients' models through a generator or through synthesised inputs, as in DENSE and its successors [19], [20], and with fusion-based methods that stitch client models together with lightweight learned adapters [21] or aggregate them through a layer-wise posterior [22], [23]. These methods are the closest to ours on the one-shot and distillation axes, and they establish that a single round can produce a usable joint model. What they do not provide is any cryptographic protection: they operate in plaintext, so the contributions each client sends are exposed.

Closing that last gap, a handful of works add privacy to one-shot federated learning, and they do so with differential privacy. FedAUXfdp distils through a pretrained feature extractor on auxiliary public data and releases the client contributions under a differentially private mechanism [24]; related approaches protect a diffusion-generated [25] or a teacher-ensemble [26] surrogate, or add noise-free differential privacy to the distillation itself [27], under the same kind of guarantee. These are the natural quantitative peers for our method, since they share its one-shot, distillation-based structure, but their privacy is lossy in exactly the sense above: the noise needed for a meaningful budget is added to the artifact that is released, and accuracy falls as the budget tightens.

The reason privacy is the binding concern here is that the artifacts federated protocols exchange leak. Sharing gradients or model updates permits reconstruction of training inputs [28], [29], [30] and supports membership and property inference [31], [32], [33], [34], [35], [36], and even released confidences enable model inversion [37], [38]. Federated distillation, which shares predictions or a public-data signal instead of gradients, was hoped to sidestep this, but a growing body of attacks shows it leaks as well: paired-logit inversion recovers images from shared logits [39], and membership can be inferred from distillation outputs and from public-dataset usage [40], [41], [42], [43], [44]. This is why we protect the contributions cryptographically rather than by perturbation, and why we measure in section VIII what the released model still reveals.

Seen together, the two strongest lines never meet. The cryptographic work that would protect contributions without loss is either multi-round encrypted training or a secure-aggregation primitive, and the one-shot distillation work that communicates in a single round is either plaintext or made private only by

TABLE I
NOTATION.

Symbol	Meaning
N	number of participating clients
$\mathcal{D}_j, n_j$	client j 's local dataset and its size $n_j = \mathcal{D}_j $
C	number of classes
α	Dirichlet concentration (label heterogeneity)
φ	frozen feature extractor (never trained or encrypted)
h_ψ	trainable classifier head, parameters ψ
f_θ	student model $h_\psi \circ \varphi$, parameters θ
θ_0	shared loss basin: common head initialization
t_j	client j 's local teacher
K, τ	distillation steps and temperature
Δ_j	displacement $\theta_j^{(K)} - \theta_0$
w_j	sample weight $n_j / \sum_i n_i$
θ^*	aggregated global model $\theta_0 + \sum_j w_j \Delta_j$
(ϵ, δ)	differential-privacy budget on Phase-0 summaries
K_{pc}	samples per class per client averaged into a prototype

lossy perturbation. The method we present occupies the empty intersection: a one-shot federated distillation whose single encrypted operation is a linear aggregation under multiparty homomorphic encryption, giving the accuracy-preserving privacy of the cryptographic line at the communication budget of the one-shot line.

III. PRELIMINARIES AND THREAT MODEL

A. Notation

Table I collects the symbols used throughout. We write client indices as $j \in \{1, \dots, N\}$ and denote by $\sigma_\tau(\cdot)$ the softmax function at temperature τ .

B. Multiparty Homomorphic Encryption

We build on the CKKS scheme [45], a ring-learning-with-errors construction [46] that performs approximate arithmetic on vectors of real numbers under encryption. CKKS packs many real slots into one ciphertext and supports two homomorphic operations of interest here: addition of two ciphertexts, and multiplication of a ciphertext by a plaintext. It is a levelled scheme, meaning each multiplication consumes one level of a finite budget; our protocol uses only additions and multiplications by public plaintext scalars, so it consumes a single level and never requires the expensive bootstrapping step [47] that refreshes a depleted budget.

A single-key scheme would require one party to hold the secret key and therefore to be trusted with decryption. We instead use the multiparty variant of CKKS [48], in which the secret key is shared among the clients through a distributed key-generation protocol that produces a collective public key without ever materialising the secret in one place. Any party, including the server, may compute on ciphertexts under the collective public key, but decryption requires the cooperation of a threshold of clients, carried out as a collective key-switching protocol. No single party, and in particular not the server, can decrypt on its own. This is precisely the property our aggregation needs: the server performs the homomorphic computation on encrypted client contributions, yet only the clients, acting together, can reveal the result.

C. Differential Privacy of Averaged Summaries

A randomised mechanism M is (ϵ, δ) -differentially private if for all adjacent inputs differing in one record and all measurable S , $\Pr[M(x) \in S] \leq e^\epsilon \Pr[M(x') \in S] + \delta$. The Gaussian mechanism achieves this for a vector-valued query by adding noise $\mathcal{N}(0, \sigma^2 I)$ with $\sigma \propto \Delta_2 \sqrt{2 \ln(1.25/\delta)}/\epsilon$, where Δ_2 is the ℓ_2 sensitivity of the query, the largest change in its output from altering one record.

The only data-derived quantities released before encryption in our protocol are per-class feature averages, and we use this structure to our advantage. If each released summary is the mean of K_{pc} clipped per-example features, then changing one example changes the mean by at most clip/K_{pc} , so the sensitivity is $\Delta_2 = \text{clip}/K_{pc}$ and the required noise scales as $\sigma \propto \text{clip}/(K_{pc} \epsilon)$. Averaging more samples per class lowers the sensitivity and hence the noise, so a client tightens its privacy budget by averaging over more of its data rather than by accepting a worse contribution. We refer to this as the averaging variant of the Gaussian mechanism and apply it to the Phase-0 prototypes in sections V-B and V-E.

D. Threat Model

The participants are N clients and a single aggregation server. The server is honest-but-curious: it follows the protocol faithfully but may attempt to infer private information from anything it observes. During aggregation the server sees only ciphertexts under the collective key, and as established in section III-B it cannot decrypt them; it is moreover excluded entirely from the shared-basin construction, which the clients carry out among themselves over secure, authenticated peer-to-peer channels. A minority of clients may likewise be honest-but-curious and may collude, but any coalition below the decryption threshold learns nothing about another client's displacement, since recovering a plaintext requires the cooperation of the threshold. This is a stronger position than secure-aggregation protocols, where the revealed sum has itself been shown to leak across rounds [49], [50], because in our one-shot protocol the server never obtains a plaintext sum at all.

Three properties follow. Raw data never leave their client. The model updates are protected by encryption alone and are therefore released to the aggregation without any accuracy-reducing perturbation. And the small per-class summaries used to form the shared basin, the only data-derived values exposed before encryption, are released under the differential-privacy mechanism of section III-C. What this model does not on its own quantify is how much an adversary can infer from the final global model, which every party receives in the clear after decryption; we measure that residual exposure directly with membership-inference attacks in section VIII. Robustness to actively malicious or Byzantine clients is outside our scope and we return to it only as a direction for future work.

IV. MOTIVATION

The privacy goal fixes the shape of the protocol before any modeling choice is made. The contributions must be combined while encrypted, in a single round, and the party that combines them must learn nothing in the process. That party is therefore

blind: it holds ciphertexts that the security of the scheme makes indistinguishable from random, and it must produce the aggregate without ever seeing a value it operates on.

Blindness is a stronger constraint than cost. A homomorphic server evaluates a fixed arithmetic circuit chosen in advance, and it cannot read a value, compare two values, or branch on the outcome of a comparison without the deep circuits, bootstrapping, or extra interaction that a one-shot, low-depth protocol rules out. Little programmability is available in practice: the server cannot cheaply decide which client to trust, which coordinates to keep, or how to precondition an update by its magnitude, because each of those decisions would have to read the encrypted data. The computation a blind server can afford is a shallow, data-independent one, and the natural such computation is a weighted sum. Whatever adaptivity the method needs must therefore live where the server is not: in the clients' local plaintext training, or in public scalars — the sample weights and the scaling coefficient — that reveal nothing about any client's data.

A blind weighted sum is only meaningful when the things being summed already agree. Averaging two networks that began from different initializations and fit different data gives a model worse than either, because their minima are not joined by a straight line and the combination lands on the ridge between two basins rather than inside one. For the server's single operation to return a usable model, the clients' models must share a loss basin: a common point they all departed from by a bounded amount, so the straight line between them stays inside one minimum. We see this directly in the experiments — remove the shared basin and the blind linear combination collapses toward chance, while restoring it makes the same operation yield a strong model (section VI-C).

Training from scratch cannot supply that common point in a single round. Independently initialized clients have no shared frame, and the methods that recover one after the fact, by matching neurons or permutations across models, must read the models — which is exactly what encryption withholds. A pretrained model supplies this frame already. Freezing the backbone fixes the representation every client sees, so each client's trainable parameters start at the same place and move only a short distance; the shared loss basin *is* the pretrained model. Fine-tuning a small set of parameters on top of a fixed, common backbone is therefore not a deployment convenience but the learning paradigm that fits a one-shot, blind, low-depth protocol, and it is what lets the encrypted object be a handful of adapter parameters rather than a whole network.

This also fixes the rest of the design. The trainable unit is a low-rank adapter and a head rather than the full model, because keeping the fine-tuned parameters few keeps the circuit shallow and the ciphertexts cheap while leaving the learning where it belongs, in the client's plaintext fine-tuning. The server's role contracts to the one thing it can soundly do — add the clients' bounded fine-tuning displacements, scaled by public weights — and that operation is a depth-one homomorphic sum, the task arithmetic of section V. The accuracy the frozen features do not already supply is then provided not by the server but by the clients, whose local fine-tuning is the source of the lift that the aggregate inherits

(section VI-D).

V. METHOD

A. Setting and Overview

We consider N clients that wish to train a shared classifier together but cannot pool their data. Client j holds a private labelled dataset $\mathcal{D}_j$ of size n_j over a common label space of C classes. The data are heterogeneous: the class proportions on each client follow a Dirichlet distribution with concentration α , and a smaller α means each client observes fewer classes and the partition is more skewed. Two constraints separate our setting from ordinary federated learning. The protocol is *one-shot*: each client uploads once and downloads once, with no iterative exchange of model updates. And privacy is *cryptographic*: a client's contribution is never revealed in the clear to the server or to any other party, and we are unwilling to buy that privacy with the accuracy loss that differential privacy incurs when it is the only line of defence.

Every client uses a model of the same form, namely a frozen feature extractor φ followed by a small trainable head h_ψ with parameters ψ , composed as $f_\theta = h_\psi \circ \varphi$. The feature extractor is either a publicly available pretrained backbone or a from-scratch architecture fixed in advance; it never changes, is never transmitted, and is never encrypted. Only the head is trained, communicated, or operated on under encryption. Keeping the trainable part this small is what makes the encrypted step inexpensive: a classifier head is a few thousand scalars, so a client's entire encrypted contribution fits in a single ciphertext, as we show in section V-D. It is natural to ask whether so small a trainable part limits accuracy; section VI shows that it does not, and that adding trainable capacity does not improve the result in this regime.

HE-IFD proceeds in three stages, illustrated in fig. 1. First, before any local training, the clients agree on a common initialization θ_0 for the head using only lightweight per-class summaries exchanged over secure peer-to-peer channels, with the server taking no part. Second, starting from θ_0 , each client distills its own local teacher into the shared head for a bounded number of steps and records the displacement it has travelled, $\Delta_j = \theta_j^{(K)} - \theta_0$. Third, the server combines the encrypted displacements into a single global model, $\theta^* = \theta_0 + \sum_j w_j \Delta_j$, using only homomorphic additions and multiplications by public scalars, after which a threshold of clients jointly decrypts the result. The three stages are the subjects of sections V-B to V-D.

What we are after is not a model that outperforms each client on that client's own dominant classes; a local specialist trained on a narrow slice of the label space is hard to beat on that slice, and we do not claim to. We are after a common model that every party can use, that classifies well across the whole label space including classes a given client never observed locally, and that is produced without any party exposing its data or its update in the clear.

B. The Shared Loss Basin

The natural one-shot recipe is to let each client train its head independently and have the server average the results. Under

Algorithm 1 One-shot federated distillation with encrypted aggregation. $\text{Enc}(\cdot)$ denotes encryption under the clients' collective CKKS key.

Require: clients $1, \dots, N$ with data $\mathcal{D}_j$; frozen extractor φ ; steps K ; temperature τ
Phase 0 — shared loss basin (peer-to-peer; server excluded)

- 1: each client computes per-class feature means through φ , optionally under (ϵ, δ) -DP
- 2: clients exchange and pool the summaries, and warm-start the head to obtain θ_0
- 3: clients run distributed key generation, yielding a collective CKKS key

Local distillation — each client j in parallel

- 4: train the local teacher t_j on $\mathcal{D}_j$; $\theta_j^{(0)} \leftarrow \theta_0$
- 5: **for** $k = 1$ to K **do**
- 6: $\theta_j^{(k)} \leftarrow \theta_j^{(k-1)} - \eta \nabla \text{KL}(\sigma_\tau(t_j) \parallel \sigma_\tau(f_{\theta_j}))$
- 7: **end for**
- 8: $\Delta_j \leftarrow \theta_j^{(K)} - \theta_0$; upload $\text{Enc}(\Delta_j)$

Encrypted aggregation — server

- 9: $\text{Enc}(\theta^*) \leftarrow \theta_0 + \sum_j w_j \cdot \text{Enc}(\Delta_j)$ ▷ PT×CT, CT+CT; depth 1
- 10: broadcast $\text{Enc}(\theta^*)$

Threshold decryption — clients

- 11: a threshold of clients jointly decrypt $\text{Enc}(\theta^*)$ to recover θ^*

heterogeneity this fails. When each client begins from its own initialization and fits its own skewed local distribution, the trained heads come to occupy different regions of parameter space, and their average lies in none of them. The per-client updates point in conflicting directions, so summing them cancels signal rather than reinforcing it, and the combined model is no better, and often worse, than its parts. We quantify this conflict directly in section VI.

We remove the conflict at its source by having every client begin from the same head initialization θ_0 , which we call the shared loss basin. The term is chosen deliberately. We do not require θ_0 to be an accurate classifier; its role is to place all clients in a common region of the loss landscape so that the directions they subsequently move in, their displacements Δ_j , live in one shared frame and add constructively rather than fighting one another. In this setting a weak but shared starting point is worth more than a strong but private one. The distinction is central to the paper, and section VI shows its two consequences in turn: without a shared basin the one-shot combination collapses, and even a deliberately weak basin is enough for the distillation that follows to yield a strong common model.

Constructing θ_0 must not leak the data we set out to protect, so clients never exchange examples. Instead, over secure peer-to-peer channels from which the server is excluded, each client computes the average feature vector of each class it holds, taken through the frozen extractor φ , and shares only these per-class summaries. The summaries are pooled into a small aligned set on which the shared head is warm-started to obtain θ_0 . Because the only quantity that leaves a client is a per-class

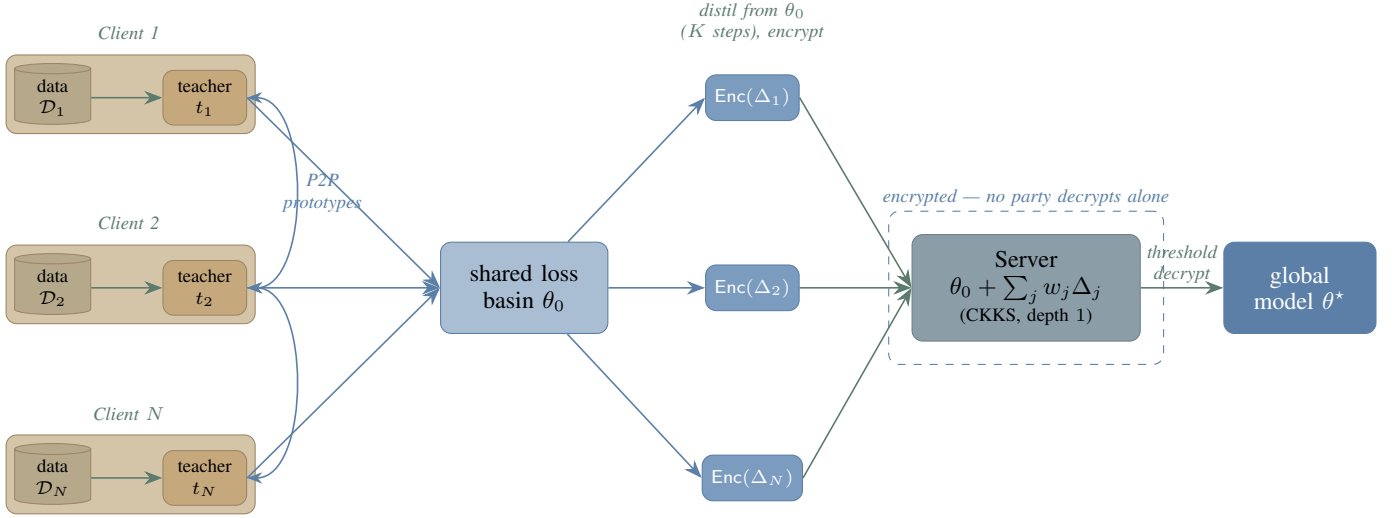


Fig. 1. HE-IFD. Each client holds private data $\mathcal{D}_j$ and trains a local teacher t_j . In Phase 0 the clients exchange per-class feature prototypes over peer-to-peer channels, with the server excluded, to agree on a shared loss basin θ_0 . Each client then distills its teacher into the shared head for a bounded number of steps and encrypts the resulting displacement Δ_j . The server's sole operation is the sample-weighted linear combination $\theta^* = \theta_0 + \sum_j w_j \Delta_j$ under multiparty CKKS (additions and public-scalar multiplications only; multiplicative depth one); it never decrypts. A threshold of clients jointly decrypts the global model θ^* .

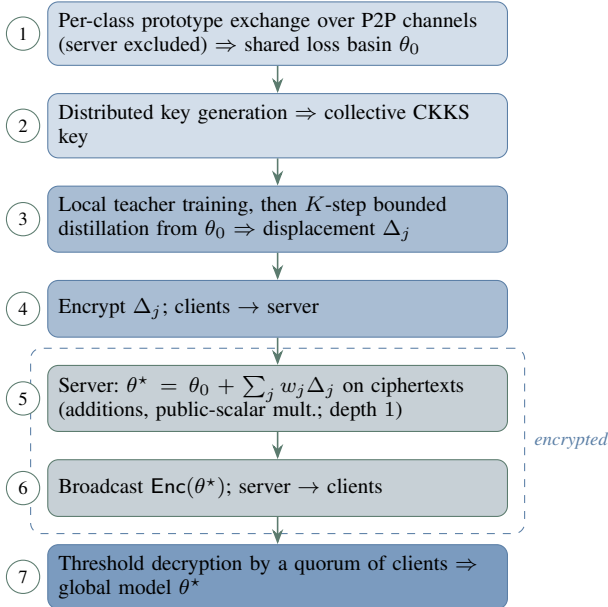


Fig. 2. Step-by-step view of the protocol. Steps 1–3 run among the clients without the server; the displacements are encrypted (step 4) and the server's single operation is the linear combination of step 5, which it performs without being able to decrypt; the global model is recovered only by a quorum of clients (step 7).

average, the leakage is small and is simple to bound: when stronger guarantees are required, the averages are released under an (ϵ, δ) differential-privacy mechanism whose noise is calibrated to the averaging sensitivity, which we detail in section V-E.

What matters for the protocol is that a shared, geometry-aligning θ_0 is produced cheaply, not how it is produced. We therefore treat the basin source as a deployment choice and evaluate several interchangeable instantiations in section VI:

a small public probe set; differentially-private per-class prototypes computed from a small fraction of local data; per-client synthetic samples; and, at the extreme, a basin formed with no labelled public data at all. Each serves the same purpose, and the protocol that follows is identical in every case.

C. Local Bounded-Trajectory Distillation

Each client first trains a local teacher t_j on its own data $\mathcal{D}_j$. The teacher is a strong specialist on the classes the client actually holds, but it is also private and idiosyncratic: it encodes that client's particular class mixture and cannot be shared or averaged with other teachers directly. The purpose of the next stage is to transfer what the teacher knows into the shared head, in a form that can be combined across clients.

Starting from the shared basin θ_0 , client j runs K steps of knowledge distillation, training the head to match its teacher's softened predictions by minimising $\text{KL}(\sigma_\tau(t_j) \parallel \sigma_\tau(f_\theta))$, where σ_τ is the softmax at temperature τ . The number of steps K is deliberately bounded and small. This is not an optimisation shortcut but a requirement of the method: a short trajectory keeps the head close to θ_0 , and therefore inside the shared basin, so that the displacement it produces is still expressed in the common frame established in section V-B. Allowed to train to convergence, each client would drift to its own local optimum and reintroduce exactly the conflict that the shared basin was constructed to remove. The temperature τ and the step budget K are the two hyperparameters of this stage; we use modest fixed values throughout and show in section VI that the gain is robust to their choice.

At the end of its trajectory the client has parameters $\theta_j^{(K)}$. Rather than the parameters themselves, the client retains and will transmit the displacement

$$\Delta_j = \theta_j^{(K)} - \theta_0, \quad (1)$$

the cumulative distance it has travelled from the shared basin. Because every client starts from the same θ_0 , these displacements are directly comparable across clients, which is what makes the linear combination in the next stage meaningful.

D. Encrypted Linear Aggregation

The clients hold a joint homomorphic-encryption key produced by a distributed key-generation protocol, so that no single party, and in particular not the server, ever holds the secret key. Each client encrypts its displacement Δ_j , the bounded trajectory of improvement it computed in section V-C; since the head is only a few thousand scalars, this fits in a single ciphertext. The server's task is to fuse these per-client trajectories into a single update of the shared model, which it does by the sample-weighted combination

$$\theta^* = \theta_0 + \sum_{j=1}^N w_j \Delta_j, \quad w_j = \frac{n_j}{\sum_i n_i}, \quad (2)$$

in which each client's trajectory contributes in proportion to the data n_j behind it, with the weights w_j public scalars. Because every Δ_j is expressed in the common frame of the shared basin (section V-B), eq. (2) is a consensus over the clients' directions of improvement rather than an average of unrelated weight vectors: the trajectories reinforce on the components where clients agree and cancel where they conflict, and the bounded, basin-anchored construction of section V-C is what keeps that consensus inside a region where the model is good.

This fusion is realised entirely with the two cheapest homomorphic operations. Forming eq. (2) requires only multiplying each ciphertext by the public scalar w_j and adding the results, with the public basin θ_0 added as a plaintext; there is no multiplication of one ciphertext by another and no bootstrapping, so the circuit has multiplicative depth one. A threshold of clients then jointly decrypts θ^* by collective key-switching, and the server never observes a plaintext update at any point. This is the design choice on which the whole approach turns: by casting aggregation as a fusion of update trajectories that is *linear in the encrypted quantities*, we obtain a server computation that maps onto the cheapest operations a levelled scheme provides, free of the encrypted nonlinearities and polynomial activations that make encrypted training expensive. We instantiate eq. (2) in a real multiparty CKKS implementation and report its measured correctness and cost in section VI.

It is worth being precise about why this works, because eq. (2) can be read too quickly. Since the weights sum to one, the displacement form is algebraically equal to a weighted average of the clients' final heads, $\theta^* = \sum_j w_j \theta_j^{(K)}$. This linearity is what keeps the operation inexpensive to encrypt. But the same average computed over heads that were trained independently, from different initializations, to convergence, is precisely the naive combination that fails in section V-B. What makes the average land in a good region here is not the averaging but the protocol around it: every $\theta_j^{(K)}$ starts from the shared basin θ_0 and moves only a bounded number of steps, so all of them remain in the common frame and their weighted

average is itself a sensible model. The contribution is therefore not a new aggregation rule but a one-shot protocol under which a linear, and hence cheaply encryptable, aggregation actually produces a strong shared model.

E. Privacy

The protocol protects privacy in two layers, applied to two different objects. The model updates themselves are protected cryptographically. The aggregation of eq. (2) runs entirely under multiparty CKKS with threshold decryption, so the displacements Δ_j are never exposed to the server or to any minority of clients, and they are protected without any perturbation: the cryptographic layer adds no accuracy cost. This is the main privacy claim of the paper and the point on which it departs from differentially private one-shot federated learning, which injects noise into the model updates that are ultimately released and therefore pays a utility tax on the very thing it ships.

The second layer concerns the only data-derived quantities that leave a client before encryption, namely the per-class feature averages used to build the shared basin in section V-B. These are released under the averaging variant of the Gaussian mechanism. Averaging K_{pc} samples per class before release lowers the sensitivity of the summary in proportion to $1/K_{pc}$, so the noise needed for an (ϵ, δ) guarantee scales as $\text{clip}/(K_{pc} \epsilon)$; a client buys privacy budget by averaging more samples rather than by degrading the model it contributes. Confining the perturbed quantity to these small summaries, and protecting the actual updates with cryptography that adds no noise, is what lets the method offer a formal (ϵ, δ) guarantee at little cost in accuracy. In section VI we trace this frontier and find that accuracy is essentially unchanged across the practical range of privacy budgets, from the non-private setting through the single-digit ϵ typical of differentially private learning, with a decline only under the most stringent budget we consider ($\epsilon = 0.5$); $K_{pc} \geq 5$ suffices there.

Two parties are assumed throughout: an honest-but-curious server that follows the protocol but may try to infer information from what it sees, and clients a minority of which may collude. The cryptographic layer defends against both, since the server only ever handles ciphertexts and no minority below the decryption threshold can recover a plaintext update. What the protocol does not by itself characterise is how much can be inferred from the *released* global model, which every party obtains. We measure that residual leakage separately through membership-inference attacks in section VIII.

F. Security Analysis

We make the privacy guarantee precise. Let the view of the honest-but-curious server during one execution be everything it receives, namely the public collective key, the public weights $\{w_j\}$ and basin θ_0 , the client ciphertexts $\{\text{Enc}(\Delta_j)\}_{j=1}^N$, and the homomorphically computed $\text{Enc}(\theta^*)$. We write $\text{view}_{\text{srv}}(\{\mathcal{D}_j\})$ for this view as a function of the clients' private datasets.

Proposition 1. *Under the IND-CPA security of the multiparty CKKS scheme and its t -out-of- N threshold decryption, for any*

honest-but-curious server colluding with up to $t - 1$ clients there is a probabilistic polynomial-time simulator $\mathcal{S}$, given only the public parameters, such that $\text{view}_{\text{srv}}(\{\mathcal{D}_j\}) \stackrel{c}{\approx} \mathcal{S}$. In particular the server's view is computationally independent of the private datasets $\{\mathcal{D}_j\}$ and of the displacements $\{\Delta_j\}$, and the server cannot recover any plaintext, including θ^ , without the cooperation of a decryption quorum.*

Proof sketch. The simulator outputs the public parameters together with N encryptions of zero under the collective key. By IND-CPA each genuine ciphertext $\text{Enc}(\Delta_j)$ is computationally indistinguishable from $\text{Enc}(0)$; a standard hybrid argument over the N ciphertexts replaces them one at a time, so the whole collection is indistinguishable from the simulated one and hence carries no information about $\{\Delta_j\}$ or the data they derive from. The aggregate $\text{Enc}(\theta^*)$ is a deterministic function of those ciphertexts and the public values $\{w_j\}, \theta_0$, so it adds nothing to the view. Finally, decryption needs t shares of the secret key, which is never reconstructed in one place; a coalition of the server and at most $t - 1$ clients holds too few shares to decrypt any ciphertext. $\square$

Proposition 1 pins down where information can and cannot flow. The server never sees plaintext data, plaintext updates, or the secret key; everything it touches is encrypted under a key it cannot use alone. Consequently the protocol exposes private information through exactly two channels, both of them intrinsic to the task rather than artifacts of our construction. The first is the Phase-0 alignment. The per-class summaries are the only data-derived quantities released before encryption, they travel over peer-to-peer channels and never reach the server, and what they reveal is precisely what the chosen alignment mechanism reveals: a per-class mean in the raw case, an (ε, δ) -bounded mean under the differentially private variant, synthetic samples under the synthetic variant, and nothing labelled at all in the no-probe case. This is a deliberate knob, traded against utility in section VI-D. The second channel is the released model itself: after threshold decryption every client holds θ^* in the clear. White-box access to the jointly built model is unavoidable in any protocol whose purpose is to hand the parties a shared model, and the residual information it carries about training data is what we quantify with membership inference in section VIII. No third channel exists.

G. Communication and Computation Cost

The protocol is inexpensive and scales well with the number of clients. Each client sends one ciphertext and receives one: because a classifier head of d parameters fits in a single CKKS ciphertext whenever d is at most the slot count, a client's upload and download are each $O(d)$ and, crucially, independent of the frozen backbone behind the head. The entire exchange is a single round. The server's work is N multiplications of a ciphertext by a public scalar followed by N ciphertext additions, an $O(Nd)$ computation of multiplicative depth one with no bootstrapping. Both the per-client cost and the round count are therefore constant in the depth of the model, in contrast to encrypted-training approaches whose cost grows

with the number of rounds and with a deep per-step circuit, and to secure-aggregation primitives that repeat over an iterative protocol.

The structure is also highly parallel. The clients' teacher training, distillation, and encryption are mutually independent and run simultaneously, and the server's aggregation is a sum, which is associative and admits a tree reduction, so end-to-end wall-clock is essentially one client's local computation plus a single aggregation regardless of N . We report measured times and communication in section VII and compare them against prior systems in section VI-H.

VI. EXPERIMENTS

Our evaluation answers, in order, what the method delivers, why each part of it is needed, what the shared basin contributes, and what privacy and communication cost the cryptographic protocol carries. Throughout, the question we are asking is not whether the global model attains the highest possible test accuracy, but whether the clients can jointly produce a single model that is stronger and more broadly capable than what any of them could obtain alone, and whether they can do so under one round of communication and accuracy-preserving cryptographic privacy.

A. Setup

We evaluate on two regimes that bracket realistic deployments. In the from-scratch regime the feature extractor is a small network trained as part of the protocol: a multilayer perceptron on MNIST and a LeNet-style convolutional network on Fashion-MNIST [51]. In the pretrained regime the feature extractor is a fixed, publicly available backbone whose features are read off without fine-tuning: a ViT-B/32 vision transformer on CIFAR-100 [52], and the RoBERTa-base and MPNet sentence encoders on AG-News. Data are partitioned across clients by a Dirichlet distribution over labels with concentration parameter α (the standard label-skew partition of federated learning: a smaller α leaves each client a few dominant classes and almost none of the rest); we report the heterogeneous ($\alpha = 0.05$), mild ($\alpha = 0.3$), and near-homogeneous ($\alpha = 1.0$) settings. Unless stated otherwise there are $N = 10$ clients, the shared basin is built from raw per-class prototypes, the local distillation runs for a small fixed number of steps K at temperature τ , and every number is averaged over three seeds. Table II summarises the configuration.

We report the test accuracy of the global model, the accuracy of the average and best local teacher as a reference for what a client obtains on its own, the accuracy of a centralized model trained on the pooled data as an upper bound, the accuracy of the shared basin θ_0 on its own, and the coverage of locally-unseen classes, defined as a client's accuracy on the classes for which it held no local examples.

B. From Local Specialists to a Shared Global Model

The main experiment shows that the protocol turns a set of narrow local teachers into a single model that is strong across the whole label space. Figure 3 shows this under

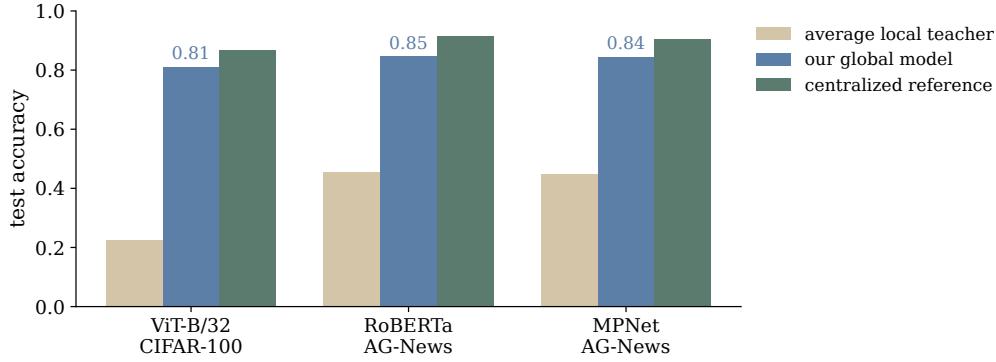


Fig. 3. The protocol turns narrow local teachers into a single, broadly capable model. Under heterogeneous data ($\alpha = 0.05$, $N = 10$), the global model (blue) reaches three to four times the accuracy of the average local teacher and comes within a few points of the centralized reference, on both vision (ViT-B/32, CIFAR-100) and language (RoBERTa, MPNet, AG-News). The frozen backbone never enters the encrypted aggregation, so the protocol and its cost are unchanged across these scales.

TABLE II

EVALUATION DESIGN. EACH AXIS ISOLATES ONE QUESTION RATHER THAN FORMING AN EXHAUSTIVE GRID; RESULTS BELOW VARY ONE AXIS AT A TIME AROUND THE DEFAULT ($N = 10$, RAW PROTOTYPES, THREE SEEDS).

Dimension	Setting	Question it probes
From-scratch vision	MNIST (MLP), Fashion-MNIST (LeNet)	behaviour without a pretrained backbone
Pretrained vision	ViT-B/32 on CIFAR-100	scaling to a strong frozen backbone
Pretrained language	RoBERTa, MPNet on AG-News	the method across modalities
Clients N	5, 10, 20, 50	scalability in participants
Heterogeneity α	0.05 (severe) to 1.0 (mild)	robustness to label skew
Shared-basin source	probe, DP prototypes, synthetic, none	interchangeability and Phase-0 privacy

heterogeneous data ($\alpha = 0.05$) for the pretrained backbones. On each task the average local teacher is weak, because under this partition a client sees only a fraction of the classes and its teacher specialises accordingly: the mean teacher accuracy is 0.23 on CIFAR-100 and below 0.46 on AG-News. The global model produced by the protocol reaches 0.81 on CIFAR-100 and 0.85 on AG-News, between three and four times the average local teacher, and lands within a few points of the centralized reference that was allowed to pool all the data. A client that participates therefore walks away with a model far better than the one it could train alone, and close to what it would have obtained had all the data been centralised, while never having shared that data.

This behaviour is not confined to one backbone scale. The same protocol, with the same bounded distillation and the same linear encrypted aggregation, holds from a small from-scratch MLP up to a frozen vision transformer and a sentence-transformer language encoder. Because the feature extractor is frozen and never enters the encrypted combination, moving to a larger backbone changes only the quality of the features the small head sees, not the protocol or its cost; the head, and hence the encrypted object, stays the same size. This is what

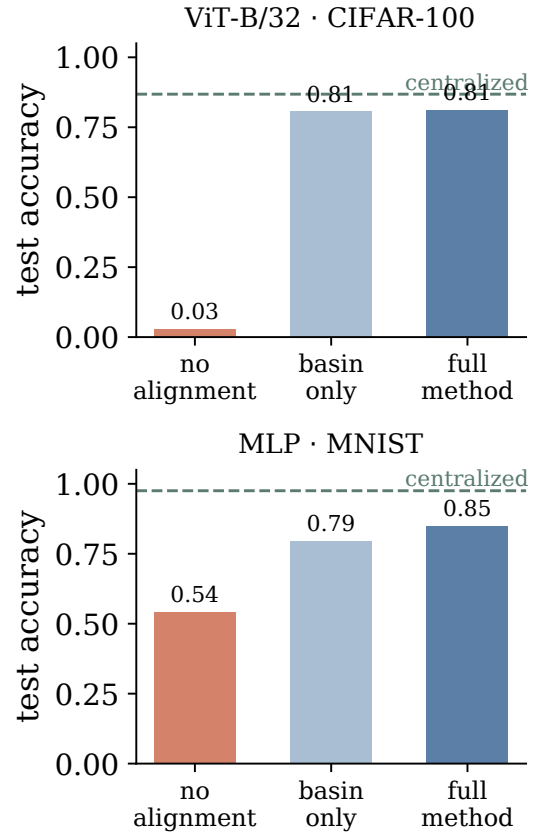


Fig. 4. Neither ingredient suffices alone ($\alpha = 0.05$). On CIFAR-100, distillation without a shared basin stays at chance, because updates from different initializations do not combine; the shared basin makes the same linear combination work. On MNIST the no-basin variant recovers partially but still trails the full method. In both cases the full method is best and approaches the centralized reference (dashed).

lets the approach scale to modern pretrained models without scaling the cryptographic work, and it is the reason a single classifier head, rather than a fine-tuned backbone, is the right unit to encrypt.

C. Necessity of the Shared Basin and Bounded Distillation

The protocol has two active parts, the shared basin and the bounded distillation, and fig. 4 shows that removing either one breaks it. We compare four points under heterogeneous data: the random initialization, distillation with no shared basin, the shared basin on its own, and the full method. On the hard task, CIFAR-100, distillation without a shared basin stays at chance (0.03): when clients begin from different initializations their bounded updates point in unrelated directions, and the linear combination of them carries no usable signal. The shared basin removes this failure by placing all clients in a common frame, after which the same linear combination yields a strong model. On the easier MNIST task the no-basin variant does not collapse completely, because distillation on ten well-separated classes recovers some structure even from a random start, but it still trails the full method by a wide margin. Neither the basin alone nor distillation alone is sufficient; their combination is what produces the result in section VI-B.

D. Distillation Lift as a Function of Basin Strength

A natural question is how much of the final accuracy is already present in the shared basin θ_0 before any distillation, and whether the distillation step is doing real work. Figure 7 answers this across the full range of basin strengths, which we vary by changing how the basin is built, from a basin formed with no labelled data at all, through differentially-private prototypes, to a rich public probe. Two facts stand out. Every point lies above the diagonal, so distillation never reduces accuracy below the basin it started from; and the vertical gap, the accuracy that distillation adds, is largest exactly where the basin is weakest, reaching more than 0.4 when the basin alone is near chance and shrinking as the basin grows strong. Where a rich basin already classifies well, distillation adds little because there is little left to add, and the final model tracks the centralized reference; where the basin is deliberately weak, distillation supplies the bulk of the accuracy. The combined mechanism, not either part, is responsible for the outcome across the whole range, and the basin's value lies in aligning the clients rather than in classifying well on its own. This also shows that the basin source is a deployment choice rather than a fixed requirement: the protocol behaves consistently whether the basin comes from a public probe, from privacy-protected prototypes, or from no labelled data at all.

Not every candidate basin source survives this test, and the one that fails is instructive. A differentially private generator can synthesize a few labelled samples per class and the basin be built from those rather than from prototypes. Once the privacy accounting is honest, however, privatizing the handful of samples a basin needs injects enough noise that the synthetic basin is markedly weaker than a prototype basin at the same budget, and the final model follows it down; prototypes, which average before they privatize and so spend the budget on a single low-sensitivity statistic, remain the better source. The same generator applied to a client's *entire* dataset, rather than to a few basin samples, is precisely the differentially private one-shot baseline of section VI-H: it must carry the whole

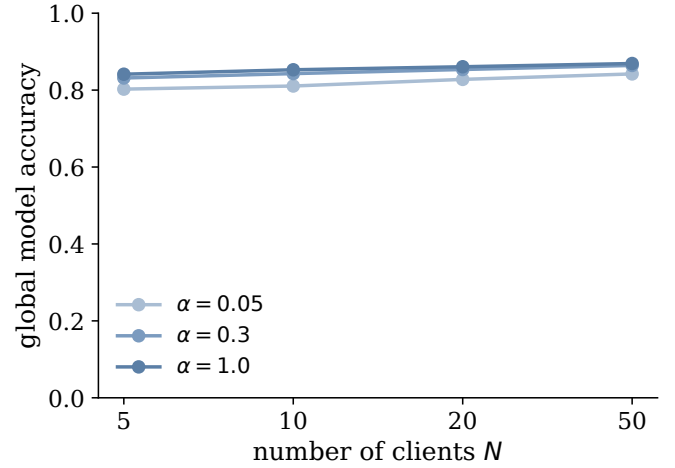


Fig. 5. The global model's accuracy is stable as the number of clients grows from 5 to 50 (ViT-B/32, CIFAR-100), at every heterogeneity level.

contribution in the clear, as a full synthetic dataset, where our method instead delegates that contribution to encryption and releases only the small private prototypes. That difference—what each design leaves exposed in the clear—is the subject of section VIII.

E. Scalability and the Effect of Trajectory Length

Two properties the protocol relies on hold up under direct test. Figure 5 varies the number of clients from five to fifty on ViT-B/32 over CIFAR-100 and shows the global model's accuracy is essentially flat, and if anything improves slightly, as clients are added; more participants do not destabilise the single linear aggregation. Figure 6 varies the length of the distillation trajectory on ResNet-18 over CIFAR-10. With the temperature that keeps the teacher's confident predictions visible, accuracy is highest at a short trajectory and declines gently as the trajectory lengthens, confirming that the bounded K is doing real work: a longer trajectory lets each client drift away from the shared basin and the aggregate degrades toward it. The figure also shows that an over-smoothed teacher signal (a high temperature) erases the gain entirely and worsens with every additional step, which is why we keep both the step budget small and the temperature low.

F. Regularizing the Aggregated Displacement

The trajectory length K is one of two levers that keep clients inside the shared basin; the other is how far the server moves the model along the combined displacement. Writing the aggregate with an explicit scaling coefficient,

$$\theta^*(\lambda) = \theta_0 + \lambda \sum_j w_j \Delta_j,$$

exposes the task-vector scaling coefficient of model merging [53]: λ sets the magnitude of the step away from the basin, and because it is a public scalar the encrypted aggregation stays at multiplicative depth one whatever value it takes. We sweep λ from 0 to 2 by re-scoring the same trajectories, with

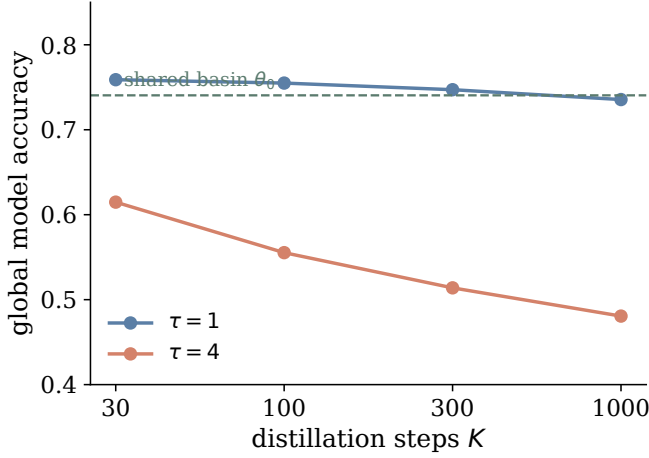


Fig. 6. The bounded trajectory matters. With a low temperature the global model stays just above the shared basin and declines gently as the trajectory lengthens (clients drift away from the basin); a high temperature erases the gain and worsens with every step (ResNet-18, CIFAR-10, $\alpha = 0.05$).

no retraining. The behaviour splits cleanly by heterogeneity. On near-homogeneous data ($\alpha = 1.0$) the optimum is exactly $\lambda = 1$: the per-client displacements already agree, and applying the combined displacement in full is best. Under severe heterogeneity ($\alpha = 0.05$) the optimum moves below 1 and accuracy declines once λ passes 1, the same shape the trajectory-length sweep shows in fig. 6. Both report one mechanism: when clients fit very different local distributions, their displacements applied at full strength carry the aggregate too far out of the common basin, and the combined step has to be regularized. Read this way λ is a drift regularizer, the continuous counterpart of the bounded step budget, and treating it as one rather than pinning it at 1 recovers a few points of accuracy in exactly the heterogeneous regime where drift is worst—on MNIST the heterogeneous optimum adds 2.4 points over $\lambda = 1$ —at no cryptographic cost, with the same downward shift of the optimum holding on the pretrained backbone. The coefficient never settles above 1, which is the operative statement: under heterogeneity the displacement should be damped toward the basin, never amplified away from it.

G. The Privacy–Utility Frontier of the Basin Prototypes

Because the model updates are protected by encryption rather than by perturbation, the only place differential privacy enters is in releasing the per-class prototypes used to build the basin. Figure 8 traces the accuracy of the global model as the privacy budget on those prototypes tightens, over the range $\epsilon \in \{0.5, 2, 8, 32\}$. At every heterogeneity level the frontier is flat across the practical privacy regime: accuracy is essentially unchanged from the non-private setting through the single-digit budgets commonly reported in differentially private learning, and begins to fall only under the most stringent setting, $\epsilon = 0.5$. The reason is the averaging structure of section III-C: a client lowers the sensitivity of its prototypes, and hence the noise it must add, by averaging over more of its own samples, so the privacy budget is bought with data rather

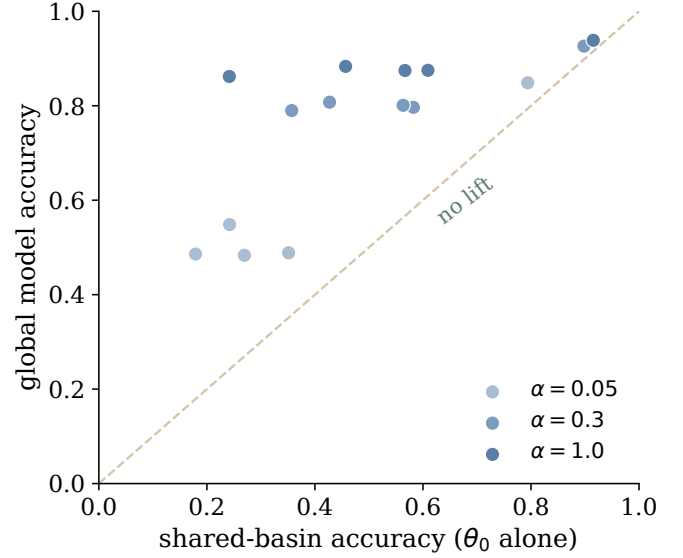


Fig. 7. Distillation never hurts and adds most where the basin is weak. Each point is a basin source at a given heterogeneity α (MNIST, $N = 10$); the horizontal axis is the accuracy of the shared basin θ_0 on its own and the vertical axis the accuracy of the resulting global model. All points lie above the no-lift diagonal, and the gap is largest when the basin is weakest.

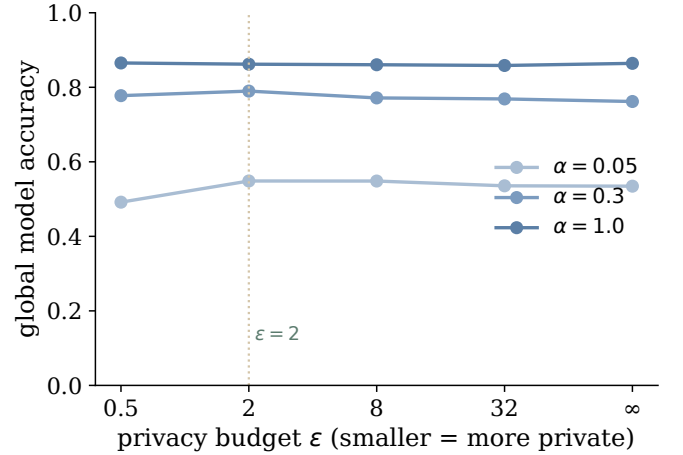


Fig. 8. The privacy–utility frontier on the basin prototypes is flat across the practical privacy regime. Tightening the differential-privacy budget on the per-class summaries through the single-digit ϵ typical of differentially private learning leaves the global model’s accuracy essentially unchanged; it falls only under the most stringent budget, $\epsilon = 0.5$ (MNIST, $N = 10$).

than with accuracy. This is the concrete form of the paper’s privacy claim, that cryptography on the updates, which adds no noise, plus differential privacy confined to the summaries delivers a formal (ϵ, δ) guarantee without the utility tax that differentially private one-shot methods pay when they perturb the released model itself.

H. Comparison with Prior Work

Table III places the method among representative federated-learning approaches along the axes that matter here: whether the protocol is genuinely one-shot, what kind of privacy it provides on the contributions, how many communication

rounds it needs, and its principal limitation relative to our goal. Existing one-shot distillation methods are either plaintext, and so offer no privacy on the contributions, or differentially private, and so pay a utility cost on the released model. The homomorphic-encryption methods in federated learning either run for many rounds, as in encrypted training, or are secure-aggregation primitives that sum model updates over an iterative protocol rather than distilling in a single shot. To our knowledge the combination we occupy, a one-shot federated distillation whose aggregation is protected by homomorphic encryption that adds no accuracy cost, has not previously been demonstrated, and this holds whether the shared basin is built from a public probe or from no labelled data at all.

Two comparisons are especially informative. Against POSEIDON, the closest cryptographic peer, the accuracy is comparable on MNIST, but POSEIDON reaches it through multi-round encrypted training that the source paper reports at roughly 1.4 hours of wall-clock, whereas our protocol completes in a single round whose server computation is milliseconds and whose communication is a few megabytes. Against the differentially private one-shot methods, the natural utility peers, the contrast is that their accuracy is bought at a privacy budget that perturbs the released model, while our encryption perturbs nothing. The communication advantage is large and structural: a full-model encrypted protocol of the kind the rejected literature assumed would move on the order of hundreds of gigabytes, whereas our single classifier-head ciphertext is a few megabytes per round (section VII). The protocol is also fully parallel across clients and its aggregation is an associative reduction (section V-G), so wall-clock does not grow with the number of participants.

[†]FuseFL's communication equals one-shot but it runs in K block-wise rounds.

[‡]Our $\alpha=0.05$ global-model accuracy on MNIST, CIFAR-100, and AG-News respectively.

I. The Incentive to Participate

It is worth being explicit about what a client gains. The global model does not, in general, beat a client's own teacher on the handful of classes that client holds in abundance; a specialist trained on a narrow distribution is hard to beat on that distribution, and our aim is not to. What the client gains is coverage of the classes it never saw locally. Under heterogeneous data the global model classifies the locally-unseen classes well above chance, whereas the client's own teacher cannot classify them at all, having never been trained on them. The value of participating is therefore the breadth that a single party cannot obtain on its own, delivered as a model every party can use, and obtained without exposing any party's data or update in the clear.

J. Scope and Limitations

Two boundaries of the approach are worth stating plainly, as both follow from well-understood properties rather than from incidental shortcomings. First, the method aggregates displacements of a head on top of a frozen extractor, so it inherits the quality of that extractor's features: it works best when the frozen representation is already linearly informative for

the task. Representations from models trained for next-token prediction are, by their nature, a weaker fit for discriminative classification than those from encoders trained bidirectionally, and we accordingly use bidirectional sentence encoders for the text tasks; the choice of a strong frozen extractor is part of the method, not a tuning detail. Second, the difficulty the shared basin addresses, conflicting client updates under heterogeneity, becomes more severe as the data grow more skewed; this is an inherent property of combining independently trained models in one shot, and the basin is our means of containing it rather than a claim that it disappears. The regime where the basin helps most is precisely the heterogeneous regime where it is most needed, and the experiments above are run in exactly that regime.

VII. EFFICIENCY

The server's only cryptographic operation is the depth-one weighted sum of eq. (2). We implement it end to end in a real multiparty CKKS protocol on the Lattigo library [54], [48] — distributed key generation, encryption under the joint key, the homomorphic aggregation, and threshold decryption by a collective key-switch — and measure what it costs. The cost is set entirely by the number of *fine-tuned* parameters that enter a ciphertext, not by the frozen backbone behind them, so a small adapter and head are cheap whatever model they sit on.

a) Parameters and setup.: All figures use ring degree 2^{14} (8192 slots per ciphertext), a modulus chain $\log Q = \{55, 45\}$ with key-switch prime $\log P = \{61\}$, and scale 2^{45} . The circuit is multiplicative *depth one* — one plaintext-scalar multiplication and a rescale, with no relinearization and no bootstrapping — and decryption is N -out-of- N (no single party can decrypt). Communication figures are exact; timings are single-run wall-clock on one core of a commodity laptop CPU (Apple M4) and are reported as indicative.

b) Communication.: A round is one upload and one download. The per-client traffic is fixed by the trainable-parameter count: a unit of up to 8192 parameters is a single 0.5 MiB ciphertext, and the 100-class vision head (76.9k parameters) spans ten. Total traffic scales linearly in the number of clients (table IV). Even the largest head at $N = 100$ moves about half a gigabyte in total, against the hundreds of gigabytes an encrypted full-model protocol would move.

c) Computation.: The server's homomorphic work is milliseconds at deployment scale and stays sub-second even at $N = 100$ on the largest head (table V). The measured operations follow simple laws: a client encrypts in about 2.2 ms per ciphertext (in parallel across clients), while the server aggregation and the threshold decryption scale as ≈ 0.39 and ≈ 0.23 ms per $N \times$ (ciphertexts/client), and the one-time distributed key generation grows linearly in N at about 1 ms per party. There is no bootstrapping anywhere in the protocol.

d) The adapter budget.: Fine-tuning a low-rank adapter rather than a linear head changes the picture only by the adapter's parameter count. A rank- r adapter on a d -dimensional layer adds $2dr$ parameters, so adapting a handful of layers adds on the order of 10^4 parameters — a small, fixed number of additional ciphertexts on top of the head.

TABLE III

COMPARISON WITH REPRESENTATIVE FEDERATED-LEARNING METHODS. REPORTED ACCURACIES ARE TAKEN VERBATIM FROM EACH SOURCE PAPER AT *its own* SETUP (DATASET, MODEL, N , HETEROGENEITY), SO THE COLUMN IS CONTEXT, NOT A LIKE-FOR-LIKE RANKING; THE PROTOCOL AXES (ONE-SHOT, PRIVACY, ROUNDS, COMMUNICATION) ARE THE CONTROLLED COMPARISON. OUR METHOD IS THE ONLY ONE THAT IS SIMULTANEOUSLY ONE-SHOT, DISTILLATION-BASED, AND PROTECTED BY ENCRYPTION THAT ADDS NO ACCURACY COST.

Method	One-shot	Privacy	Rounds	Reported acc. (own setup)	Principal limitation vs. our goal
FedDF [18]	no	none	~ 100	rounds-to-target (CIFAR-10)	many rounds; no privacy
FuseFL [21]	no [†]	none	K blocks	84.3% (CIFAR-10)	K rounds; no privacy
DENSE [19]	yes	none	1	data-free (CIFAR-10)	no privacy on contributions
FedLPA [22]	yes	none	1	85.8% (MNIST)	no privacy; needs Fisher info
FedKT [26]	yes	DP-capable	1	90.5% (MNIST)	lossy when private
FedAUXfdp [24]	yes	DP (lossy)	1	75.2% (CIFAR-10, $\epsilon=0.5$)	accuracy falls with budget
FedDiff [25]	yes	DP (lossy)	1	27.8% (CIFAR-10, $\epsilon=10$)	accuracy falls with budget
POSEIDON [4]	no	HE (no noise)	many	89.9% (MNIST, ~ 1.4 h)	multi-round; encrypted training
HE secure agg. [5]	no	HE	per round	protocol only	aggregation, not distillation
HE-IFD (ours)	yes	HE (no noise)	1	84.9 / 81.1 / 84.6% [‡]	—

TABLE IV

COMMUNICATION PER ROUND (MULTIPARTY CKKS, RING 2^{14}). PER-CLIENT TRAFFIC IS FIXED BY THE TRAINABLE UNIT; TOTALS SCALE LINEARLY IN N . EACH CIPHERTEXT IS 0.5 MiB.

Trainable unit	ct/client	up=down/client	$N=10$	total up=down $N=50$	$N=100$
text head, 4-class (3.1k)	1	0.50 MiB	5.0 MiB	25 MiB	50 MiB
vision head, 100-class (76.9k)	10	5.0 MiB	50 MiB	250 MiB	500 MiB

TABLE V

COMPUTATION TIME (MS, ONE CPU CORE). DKG IS A ONE-TIME SETUP; THE ENCRYPT FIGURE IS PER CLIENT (CLIENTS ENCRYPT IN PARALLEL); AGGREGATION AND DECRYPTION SCALE LINEARLY IN $N \times \text{CIPHERTEXTS}$.

Trainable unit	N	DKG	encrypt/client	aggregate	decrypt
text head, 4-class	10	13	2.3	3.9	2.4
text head, 4-class	100	96	2.2	38.9	23.2
vision head, 100-class	10	11	21.5	39.0	24.6
vision head, 100-class	100	98	21.7	382.2	216.5

The encrypted object stays a few megabytes per client, the circuit stays depth one, and the server cost grows only with that ciphertext count, never with the size of the frozen backbone the adapter sits in.

e) Correctness.: The decrypted aggregate matches the plaintext computation to a relative ℓ_2 error between 1.4×10^{-9} ($N = 5$) and 2.6×10^{-8} ($N = 100$). This is the encoding precision of CKKS at scale 2^{45} ; it grows mildly with N as ciphertext additions and the key-switch smudging noise accumulate, and it stays several orders of magnitude below any tolerance the decoded model is sensitive to. Because the circuit is depth one, the error is not amplified.

f) Comparison with prior cryptographic federated learning.: The contrast with prior homomorphic federated learning is structural, not incremental. Encrypted-training methods such as POSEIDON [4] reach comparable accuracy but through many rounds of homomorphic computation that the source paper reports at roughly 1.4 hours of wall-clock; secure-aggregation primitives [5] sum updates once per round inside an iterative protocol. Our protocol runs in a single round whose server computation is milliseconds and whose communication is a few megabytes, because the only thing encrypted is a

bounded fine-tuning displacement of a small adapter, combined at multiplicative depth one. The broader method-level comparison is in table III.

VIII. PRIVACY ANALYSIS

The cryptographic and differential-privacy guarantees of section V-E protect the contributions and the basin summaries, not the final model, which every party obtains in the clear after decryption. This is not a gap a stronger mechanism could close. Any protocol that lets separate parties build a common model must move knowledge between them, and a model that is useful to a client—one that classifies categories the client never saw locally—necessarily encodes information about the data that taught it those categories. No method that returns a shared, useful model can therefore promise that nothing about any party's data is ever inferable from it. The meaningful question is not whether residual leakage is zero, which it cannot be, but how large an attack surface a protocol exposes on the way to the model, and the design of section V is built to make that surface as small as a one-shot federated method can.

A. What each protocol leaves exposed

Read as attack surfaces, the alternatives differ by orders of magnitude. Multi-round federated learning exposes the most: every round publishes a fresh update computed on private data, so an honest-but-curious server or peer accumulates a long sequence of views of each client over the course of training, and the membership-inference literature targets exactly this repetition [31], [32]. One-shot methods that lean on differential privacy expose, in a single object, essentially the whole of a client's local knowledge: a differentially private one-shot pipeline that synthesizes data ships a full synthetic copy of each client's dataset in the clear, and one that perturbs a model ships the noised model itself. HE-IFD exposes the least it can while still producing a shared model. The per-client displacements are encrypted under the joint key and never appear in the clear; the only object any party receives is the single decrypted aggregate; and the only signal released in the open is the small set of per-class prototypes, themselves under differential privacy. The bulk of every contribution is delegated

TABLE VI
MEMBERSHIP INFERENCE ON THE PHASE-0 PROTOTYPE CHANNEL, AREA UNDER THE ROC CURVE. A RAW PROTOTYPE RELEASE LEAKS (MOST AT THE NEAR-HOMOGENEOUS SETTING, WHERE PROTOTYPES ARE MORE INFORMATIVE); APPLYING THE AVERAGING-VARIANT DIFFERENTIAL-PRIVACY BUDGET COLLAPSES THE SIGNAL TO THE CHANCE LEVEL (0.5) AT $\epsilon \leq 8$ ON BOTH MODALITIES.

Backbone	α	raw	$\epsilon=8$	$\epsilon=2$
ViT-B/32 (CIFAR-100)	0.05	0.97	0.57	0.53
	1.0	1.00	0.60	0.56
RoBERTa (AG-News)	0.05	0.53	0.49	0.49
	1.0	0.64	0.49	0.49

to encryption, and what remains in the open is one aggregate and a handful of private summaries.

B. Measured residual on the open channels

We measure what remains with membership-inference attacks [33], [34], [35] across the surfaces an adversary could realistically observe in our protocol: the released model, queried externally or by a fellow client who brings its own data and the shared prototypes as a prior, and the prototype channel itself. We include the attacks tailored to the federated-distillation setting [40], [41], [43], [42], and score by area under the ROC curve, with the true-positive rate at a low fixed false-positive rate alongside.

On the released model the leakage stays small. On the language task the strongest shadow-model attack sits at the level of random guessing across heterogeneity levels, so a participant learns essentially nothing about another’s membership from the shared model beyond what any usable classifier of that data would reveal. The prototype channel is the one place a *raw* release leaks—unsurprisingly, since a raw prototype is an average of real features—and it is exactly the place differential privacy applies. Table VI shows that the membership signal on the prototypes collapses to the level of guessing once the budget is applied: at $\epsilon = 8$ and below the area under the curve returns to about one half on both the vision and the language backbone. The in-the-clear channel is closed by the same averaging-variant accounting that leaves accuracy flat (section VI-G).

C. The inherent limit

We do not claim the released model leaks nothing. A model that hands every client coverage of classes it never observed must carry information about the data that supplied that coverage, and an adversary with strong priors about a client’s distribution can recover some of it; this is a property of useful shared models, not of our protocol in particular. What the protocol guarantees is that this residual is the *entire* surface. Because the contributions are never exposed, the leakage a client faces is bounded by what one shared model inherently reveals—not multiplied across the rounds of an iterative protocol, and not handed over wholesale as a released synthetic dataset or a perturbed model. Driving the exposed surface down to that inherent floor, rather than promising to remove a leak that cannot be removed when parties set out to

build a common model from their data, is the privacy goal the design is built to meet, and the measurements above show it reaches that floor on the one channel that is open.

IX. CONCLUSION

We presented HE-IFD, a one-shot federated distillation in which the server’s only operation on client contributions is a linear aggregation computed under multiparty homomorphic encryption at multiplicative depth one. The construction gives the contributions cryptographic privacy at no accuracy cost, at the communication budget of a single round, and because the feature extractor is frozen and stays outside the encrypted combination, the protocol and its cost are unchanged from small networks to frozen pretrained backbones. The element that makes a one-shot linear aggregation succeed under heterogeneity is the shared loss basin, a cheap and possibly weak common initialization whose purpose is to align the clients’ updates rather than to classify well. Across vision and language tasks the method yields a common model far stronger than any client’s own and close to a centralized one, a real multiparty CKKS implementation confirms the encrypted aggregation at a few megabytes per round, and the accuracy cost of differentially private basin summaries is negligible across the practical range of privacy budgets. Extending the approach to actively malicious participants, where verifiable homomorphic encryption [55], [56] could certify that the server computed the aggregation honestly, and tightening the analysis of what the released model itself reveals, are natural directions for future work.

REFERENCES

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *AISTATS*, 2017.
- [2] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, “Practical secure aggregation for privacy-preserving machine learning,” in *ACM CCS*, 2017, pp. 1175–1191.
- [3] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, “Deep learning with differential privacy,” in *Proceedings of the 23rd ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016, pp. 308–318. [Online]. Available: <https://arxiv.org/abs/1607.00133>
- [4] S. Sav, A. Pyrgelis, J. R. Troncoso-Pastoriza, D. Froelicher, J.-P. Bossuat, J. Sá Sousa, and J.-P. Hubaux, “POSEIDON: Privacy-preserving federated neural network learning,” in *NDSS*, 2021.
- [5] Kemmaka and Tran, “Hyb-Agg: Communication-efficient secure aggregation via hybrid multi-key homomorphic encryption and masking,” *arXiv:2511.23252*, 2025.
- [6] C. Zhang, S. Li, J. Xia, W. Wang, F. Yan, and Y. Liu, “Batchcrypt: Efficient homomorphic encryption for cross-silo federated learning,” in *USENIX ATC*. USENIX Association, 2020, pp. 493–506.
- [7] W. Jin, Y. Yao, S. Han, J. Gu, C. Joe-Wong, S. Ravi, S. Avestimehr, and C. He, “Fedml-he: An efficient homomorphic-encryption-based privacy-preserving federated learning system,” in *NeurIPS*, vol. 36, 2023.
- [8] X. Wei, R. Huang, L. Lei, and J. Wu, “Fedshe-cq: A communication-efficient homomorphic encryption framework for federated learning,” *Computer Networks*, vol. 260, p. 111813, 2025.
- [9] K. Garimella, N. K. Jha, and B. Reagen, “Sisyphus: A cautionary tale of using low-degree polynomial activations in privacy-preserving deep learning,” in *PPML Workshop, CCS*, 2021.
- [10] M. Baruch, N. Drucker, L. Greenberg, and G. Moshkovich, “A methodology for training homomorphic encryption friendly neural networks,” in *ACNS Workshops*, ser. Lecture Notes in Computer Science, vol. 13285. Springer, 2022, pp. 536–553.

- [11] P. Agamennone, "Polynomial approximations of relu for secure cnn inference in homomorphic encryption environments," *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. E108.A, no. 7, 2025.
- [12] F. Al Hossain and T. Rahman, "A training framework for optimal and stable training of polynomial neural networks," *arXiv preprint arXiv:2505.11589*, 2025.
- [13] A. Ibarrondo and M. Önen, "FHE-compatible batch normalization for privacy-preserving deep learning," in *Privacy in Machine Learning Workshop (PriML), NeurIPS*, 2021. [Online]. Available: <https://www.eurecom.fr/en/publication/5626>
- [14] N. Guha, A. Talwalkar, and V. Smith, "One-shot federated learning," *arXiv preprint arXiv:1902.11175*, 2019. [Online]. Available: <https://arxiv.org/abs/1902.11175>
- [15] J. Wang *et al.*, "Towards one-shot federated learning: Advances, challenges, and future directions," *arXiv preprint arXiv:2505.02426*, 2025.
- [16] D. Li and J. Wang, "FedMD: Heterogenous federated learning via model distillation," *arXiv preprint arXiv:1910.03581*, 2019. [Online]. Available: <https://arxiv.org/abs/1910.03581>
- [17] S. Itahara, T. Nishio, Y. Koda, M. Morikura, and K. Yamamoto, "DS-FL: Distillation-based semi-supervised federated learning for medical image classification," in *IEEE ICC*, 2021.
- [18] T. Lin, L. Kong, S. U. Stich, and M. Jaggi, "Ensemble distillation for robust model fusion in federated learning," in *NeurIPS*, vol. 33, 2020. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/18df51b97ccd68128e994804f3eccc87-Abstract.html>
- [19] J. Zhang, C. Chen, B. Li, L. Lyu, S. Wu, S. Ding, C. Shen, and C. Qi, "DENSE: Data-free one-shot federated learning," in *NeurIPS*, vol. 35, 2022, pp. 21414–21428. [Online]. Available: <https://arxiv.org/abs/2112.12371>
- [20] R. Dai, Y. Zhang, A. Li, T. Liu, X. Yang, and B. Han, "Enhancing one-shot federated learning through data and ensemble co-boosting," in *ICLR*, 2024.
- [21] Z. Tang, Y. Zhang, P. Dong, Y.-m. Cheung, A. C. Zhou, B. Han, and X. Chu, "Fusefl: One-shot federated learning through the lens of causality with progressive model fusion," in *NeurIPS*, 2024.
- [22] X. Liu, L. Liu, F. Ye, Y. Shen, X. Li, L. Jiang, and J. Li, "FedLPA: One-shot federated learning with layer-wise posterior aggregation," in *NeurIPS*, 2024.
- [23] Y. Zhang *et al.*, "Federated one-shot learning with data-free distillation," in *NeurIPS*, 2024.
- [24] H. Hoech, R. Rischke, K. Mueller, and W. Samek, "FedAUXfdp: Differentially private one-shot federated distillation," in *IJCAI Workshop on Federated Learning*, 2022.
- [25] M. Mendieta, G. Sun, and C. Chen, "Navigating heterogeneity and privacy in one-shot federated learning with diffusion models," in *WACV*, 2025, pp. 2601–2610. [Online]. Available: https://openaccess.thecvf.com/content/WACV2025/html/Mendieta_Navigating_Heterogeneity_and_Privacy_in_One-Shot_Federated_Learning_with_Diffusion_Models_WACV_2025_paper.html
- [26] Q. Li, B. He, and D. Song, "Practical one-shot federated learning for cross-silo setting," in *IJCAI*, 2021, pp. 1484–1490.
- [27] L. Sun and L. Lyu, "Federated model distillation with noise-free differential privacy," in *IJCAI*, 2021, pp. 1563–1569.
- [28] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *NeurIPS*, vol. 32, 2019.
- [29] D. I. Dimitrov, M. Baader, and M. Vechev, "SPEAR: Exact gradient inversion of batches in federated learning," in *NeurIPS*, vol. 37, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/c13cd7feab4beb1a27981e19e2455916-Abstract-Conference.html
- [30] V. Carletti, P. Foggia, C. Mazzocca, G. Parrella, and M. Vento, "SoK: Gradient inversion attacks in federated learning," in *34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association, 2025. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity25/presentation/carletti>
- [31] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *IEEE S&P*, 2017, pp. 3–18. [Online]. Available: <https://arxiv.org/abs/1610.05820>
- [32] M. Nasr, R. Shokri, and A. Houmansadr, "Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning," in *IEEE S&P*, 2019, pp. 739–753.
- [33] S. Yeom, I. Giacomelli, M. Fredrikson, and S. Jha, "Privacy risk in machine learning: Analyzing the connection to overfitting," in *31st IEEE Computer Security Foundations Symposium (CSF)*, 2018, pp. 268–282. [Online]. Available: <https://arxiv.org/abs/1709.01604>
- [34] A. Salem, Y. Zhang, M. Humbert, P. Berrang, M. Fritz, and M. Backes, "ML-Leaks: Model and data independent membership inference attacks and defenses on machine learning models," in *Network and Distributed System Security Symposium (NDSS)*, 2019. [Online]. Available: <https://arxiv.org/abs/1806.01246>
- [35] N. Carlini, S. Chien, M. Nasr, S. Song, A. Terzis, and F. Tramèr, "Membership inference attacks from first principles," in *IEEE S&P*, 2022, pp. 1897–1914. [Online]. Available: <https://arxiv.org/abs/2112.03570>
- [36] R. Kerkouche, G. Mema, M. Fritz, J. Ramon, and N. Samarin, "Client-specific property inference against secure aggregation in federated learning," in *IEEE S&P*, 2023. [Online]. Available: <https://arxiv.org/abs/2303.03908>
- [37] M. Fredrikson, S. Jha, and T. Ristenpart, "Model inversion attacks that exploit confidence information and basic countermeasures," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2015, pp. 1322–1333. [Online]. Available: <https://dl.acm.org/doi/10.1145/2810103.2813677>
- [38] B. Hitaj, G. Ateniese, and F. Pérez-Cruz, "Deep models under the GAN: Information leakage from collaborative deep learning," in *ACM CCS*, 2017, pp. 603–618.
- [39] H. Takahashi, J. Liu, and Y. Liu, "Breaching FedMD: Image recovery via paired-logits inversion attack," in *CVPR*, 2023, pp. 12 198–12 207.
- [40] Z. Yang, Y. Zhao, and J. Zhang, "FD-Leaks: Membership inference attacks against federated distillation learning," in *APWeb-WAIM*, ser. Lecture Notes in Computer Science, vol. 13423. Springer, 2023, pp. 364–378.
- [41] X. Wang, L. Wu, and Z. Guan, "GradDiff: Gradient-based membership inference attacks against federated distillation with differential comparison," *Information Sciences*, vol. 658, p. 120068, 2024.
- [42] M. Jagielski, M. Nasr, K. Lee, C. Choquette-Choo, N. Carlini, and F. Tramèr, "Students parrot their teachers: Membership inference on model distillation," in *NeurIPS*, vol. 36, 2023.
- [43] Y. Gu and Y. Bai, "LDIA: Label distribution inference attack against federated learning in edge computing," *Journal of Information Security and Applications*, vol. 74, p. 103475, 2023. [Online]. Available: <https://dl.acm.org/doi/10.1016/j.jisa.2023.103475>
- [44] H. Shi, T. Ouyang, and A. Wang, "Unveiling client privacy leakage from public dataset usage in federated distillation," *PoPETs*, vol. 2025, no. 4, pp. 201–215, 2025.
- [45] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *ASIACRYPT*, 2017, pp. 409–437.
- [46] V. Lyubashevsky, C. Peikert, and O. Regev, "On ideal lattices and learning with errors over rings," *Journal of the ACM*, vol. 60, no. 6, pp. 43:1–43:35, 2013.
- [47] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song, "Bootstrapping for approximate homomorphic encryption," in *Advances in Cryptology – EUROCRYPT 2018*, ser. Lecture Notes in Computer Science, vol. 10820. Springer, 2018, pp. 360–384. [Online]. Available: <https://eprint.iacr.org/2018/153>
- [48] C. Mouchet, J. R. Troncoso-Pastoriza, J.-P. Bossuat, and J.-P. Hubaux, "Multiparty homomorphic encryption from ring-learning-with-errors," *PoPETs*, vol. 2021, no. 4, pp. 291–311, 2021.
- [49] J. So, R. E. Ali, B. Güler, J. Jiao, and A. S. Avestimehr, "Securing secure aggregation: Mitigating multi-round privacy leakage in federated learning," in *AAAI*, vol. 37, no. 8, 2023, pp. 9925–9933.
- [50] R. Kerkouche, G. Ács, and M. Fritz, "Client-specific property inference against secure aggregation in federated learning," in *WPES*. ACM, 2023, pp. 15–30.
- [51] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," 2017. [Online]. Available: <https://arxiv.org/abs/1708.07747>
- [52] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009.
- [53] G. Ilharco, M. T. Ribeiro, M. Wortsman, L. Schmidt, H. Hajishirzi, and A. Farhadi, "Editing models with task arithmetic," in *ICLR*, 2023.
- [54] C. Mouchet, J.-P. Bossuat, J. Troncoso-Pastoriza, and J.-P. Hubaux, "Lattigo v5: A multiparty homomorphic encryption library in Go," in *WAHC*, 2021.
- [55] A. Viand, C. Knabenhans, and A. Hithnawi, "SoK: Fully homomorphic encryption compilers and verifiable computation," in *IEEE S&P*, 2023. [Online]. Available: <https://arxiv.org/abs/2301.07041>
- [56] S. Atapoor, K. Bagheri, H. V. L. Pereira, and J. Spiessens, "Verifiable FHE via lattice-based SNARKs," *Cryptology ePrint Archive, Paper 2024/582*, 2024. [Online]. Available: <https://eprint.iacr.org/2024/582>



HALİL İBRAHİM KANPAK received his B.Sc. degree in Mathematics from Koç University. He is currently pursuing his Ph.D. at Koç University, where he is a member of the Cryptography, Cyber Security & Privacy Research Group. His research interests include cryptography and related areas in Deep Learning.



Asst. Prof. SİNEM SAV received her Ph.D. in Computer and Communication Sciences from École Polytechnique Fédérale de Lausanne (EPFL). She also holds M.Sc. and B.Sc. degrees in Computer Engineering from Bilkent University, where she currently serves as an Assistant Professor in the Computer Engineering department. Her research interests include privacy-preserving machine learning and applied cryptography.



Prof. ALPTEKİN KÜPÇÜ received his Ph.D. degree from Brown University Computer Science Department in 2010. Since then, he has been working as a faculty member at Koç University, leading the Cryptography, Cyber Security & Privacy Research Group that he founded. He is an ACM and IEEE Senior Member, and a co-founder of Xtinge Technology Inc.